Introductions

Introduction to Operating Systems, Fall 2024

FIT-HCMUS

Logistics

Instructor: Trần Trung Dũng

TAs: {ctlinh, ntquan}@fit.hcmus.edu.vn

More information (including office hours, email addresses, etc.)

ttdung@fit.hcmus.edu.vn

If you need help outside of class hours, use email

Start subject line with [HĐH]

Emails without this tag will likely be ignored

Course Overview

Goals:

Understanding of OS and the OS/architecture interface/interaction

Learn a little bit about distributed OS'es if having time

Prerequisites:

What to expect:

We will cover core concepts and issues in lectures

In sections, you and your TA will practice and talk about the project

3 projects

1 Midterm and 1 Final

Warning

We will take it serious if cheating!

Textbook and Topics

Text: Operating System Concepts (8th Edition), Silberschatz and Galvin.

Reference book: Modern Operating Systems, Andrew S. Tanenbaum 3rd edition.

Topics

- Architecture
- Processes and threads
- Processor scheduling
- Synchronization
- Virtual memory
- File systems
- I/O systems

Grading

Rough guideline

1 Midterm 20%

Midterms will be held in common at class

1 Final 50%

Quiz (10%)

Lab (30%)

Remember, this is just a rough guideline

Grading should really be based on how much you have learned from the course

Any concrete thing that you do towards convincing me of this may help

For example, showing up at lab hours and participating in class will likely help

Today

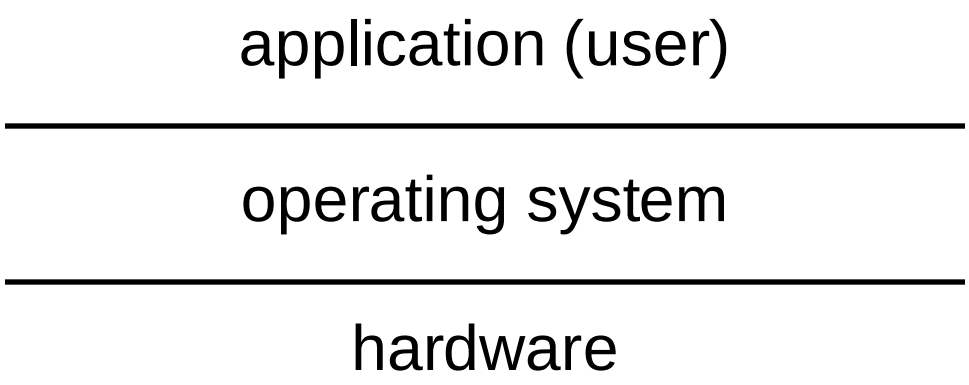
What is an Operating System?

Major OS components

A little bit of history

Silberschatz Chapter 1 & 3 (3.1-3.4)

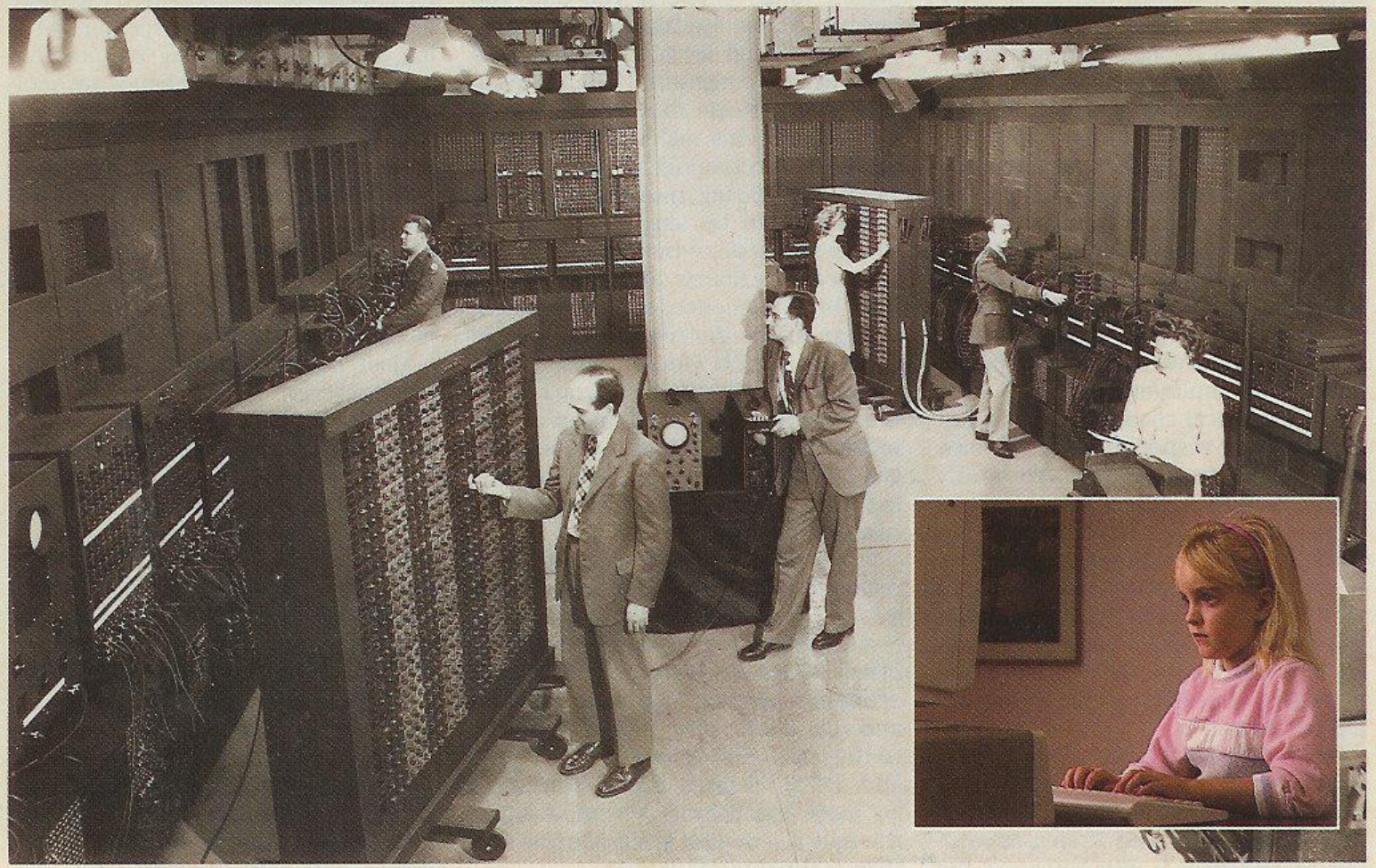
What Is An Operating System?



A software layer between the hardware and the application programs/users which provides a *virtual machine* interface: easy and safe

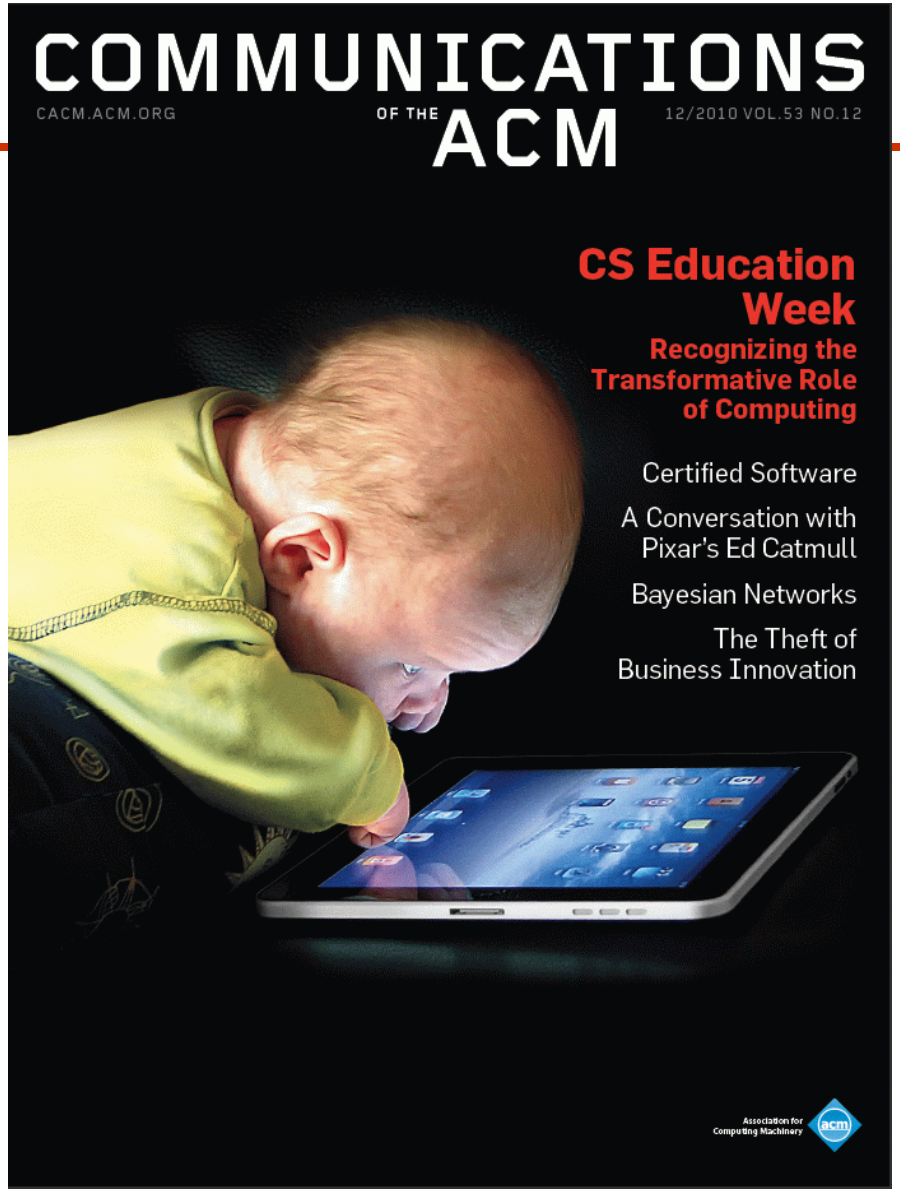
A *resource manager* that allows programs/users to share the hardware resources: fair and efficient

A set of utilities to simplify application development

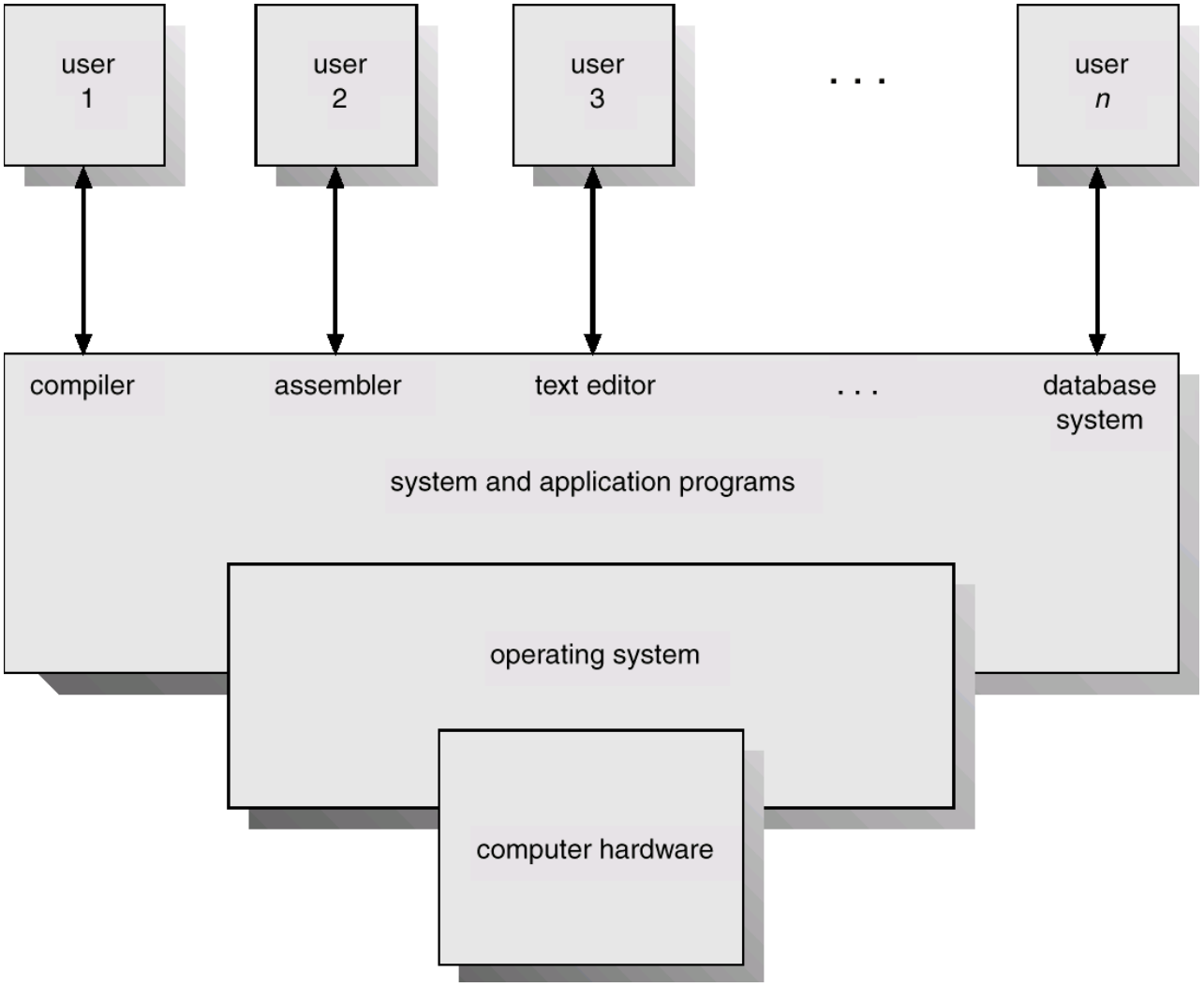


TRAINING necessary to operate a computer has been drastically reduced as a result of advances in both hardware and software. Only highly trained operators could run the first computers, which were developed some 40 years ago. Today's personal computers (which surpass those first machines in both memory and computing power) can be operated by a child.

58 SCIENTIFIC AMERICAN *May 1989*


COMMUNICATIONS
OF THE ACM
CACM.ACM.ORG
12/2010 VOL.53 NO.12
CS Education Week
Recognizing the Transformative Role of Computing
Certified Software
A Conversation with Pixar's Ed Catmull
Bayesian Networks
The Theft of Business Innovation
Association for Computing Machinery
acm

Abstract View of System Components



Why Do We Want An OS?

Benefits for application writers

- Easier to write programs
  - See high level abstractions instead of low-level hardware details
    - E.g. files instead of disk blocks
- Portability

Benefits for users

- Easier to use computers
  - Can you imagine trying to use a computer without the OS?
- Safety
  - OS protects programs from each other
  - OS protects users from each other

Mechanism And Policy

application (user)

operating system: *mechanism+policy*

hardware

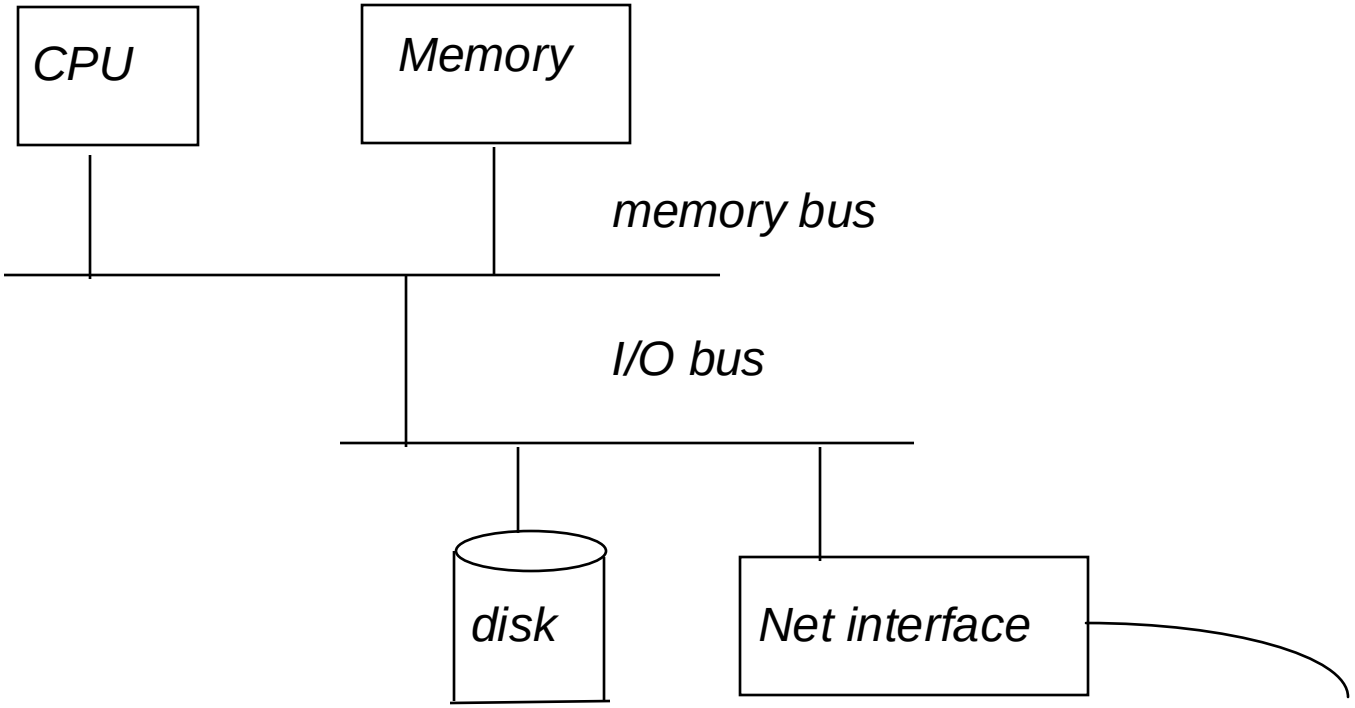
Mechanisms: data structures and operations that implement an abstraction (e.g. the buffer cache)

Policies: the procedures that guides the selection of a certain course of action from among alternatives (e.g. the replacement policy for the buffer cache)

Want to separate mechanisms and policies as much as possible

Different policies may be needed for different operating environments

Basic computer structure



System Abstraction: Processes

A process is a system abstraction:
illusion of being the only job in the system

user: *run application*

---

operating system: *process*

---

hardware: *computer*

create, kill processes,
inter-process comm.

Multiplexing resources

Processes: Mechanism and Policy

Mechanism:

Creation, destruction, suspension, context switch, signaling, IPC, etc.

Policy:

Minor policy questions:

- Who can create/destroy/suspend processes?
- How many active processes can each user have?

Major policy question that we will concentrate on:

- How to share system resources between multiple processes?
- Typically broken into a number of orthogonal policies for individual resource such as CPU, memory, and disk.

Processor Abstraction: Threads

A thread is a processor abstraction: illusion of having 1 processor per execution context

application: *execution context*

create, kill, synch.

operating system: *thread*

context switch

hardware: *processor*

Threads: Mechanism and Policy

Mechanism:

Creation, destruction, suspension, context switch, signaling, synchronization, etc.

Policy:

How to share the CPU between threads from different processes?

How to share the CPU between threads from the same process?

How can multiple threads synchronize with each other?

How to control inter-thread interactions?

Can a thread murder other threads at will?

Memory Abstraction: Virtual memory

Virtual memory is a memory abstraction:
illusion of large contiguous memory, often more memory than physically available

application: *address space*

virtual addresses

operating system: *virtual memory*

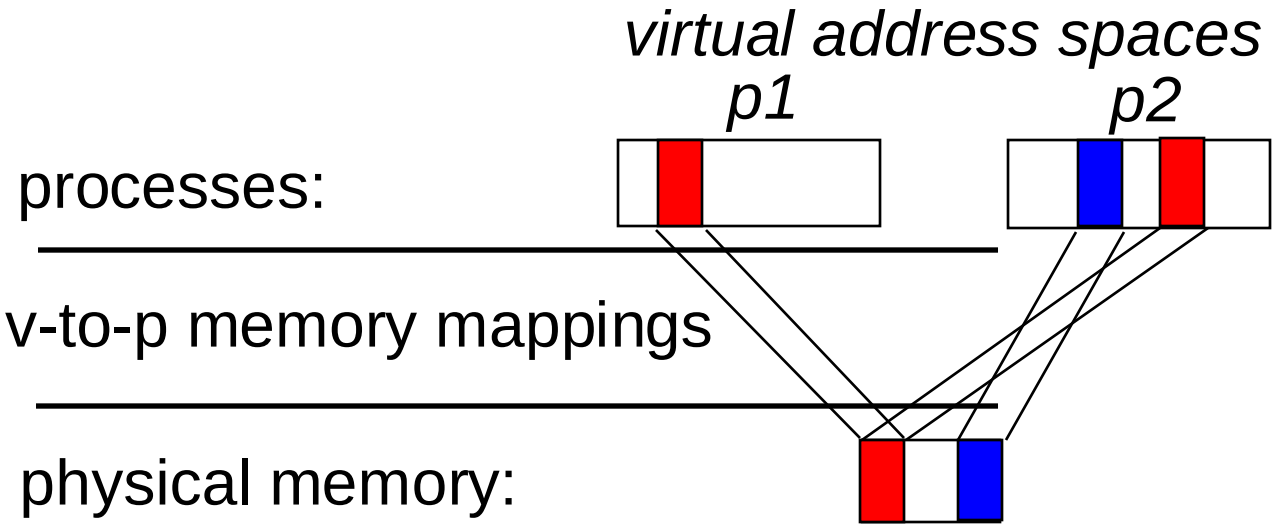
physical addresses

hardware: *physical memory*

Virtual Memory: Mechanism

Mechanism:

Virtual-to-physical memory mapping, page-fault, etc.



Virtual Memory: Policy

Policy:

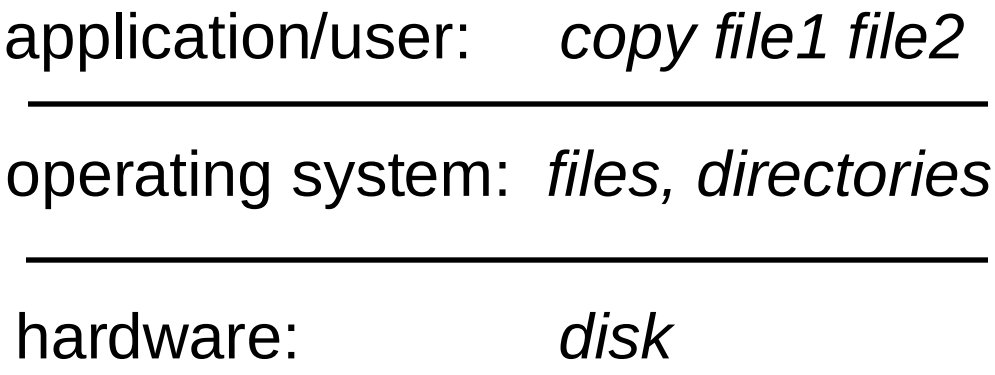
How to multiplex a virtual memory that is larger than the physical memory onto what is available?

How should physical memory be allocated to competing processes?

How to control the sharing of a piece of physical memory between multiple processes?

Storage Abstraction: File System

A file system is a storage abstraction: illusion of structured storage space



naming, protection, operations on files

operations on disk blocks...

File System

Mechanism:

File creation, deletion, read, write, file-block-to-disk-block mapping, file buffer cache, etc.

Policy:

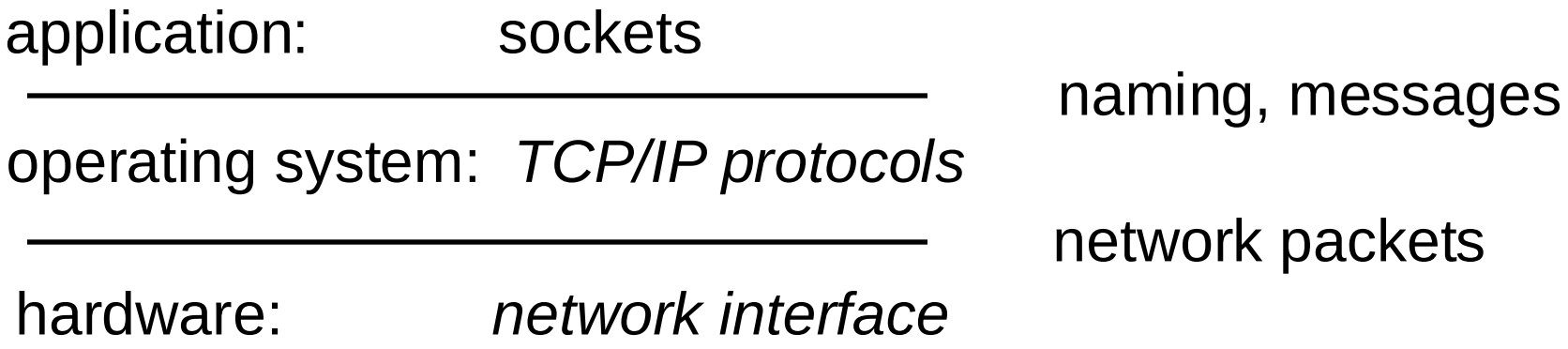
Sharing vs. protection?

Which block to allocate?

File buffer cache management?

Communication Abstraction: Messaging

Message passing is a communication abstraction:
illusion of reliable (sometime ordered) transport



Message Passing

Mechanism:

Send, receive, buffering, retransmission, etc.

Policy:

Congestion control and routing

Multiplexing multiple connections onto a single NIC

Character & Block Devices

The device interface gives the illusion that devices support the same API – character stream and block access

application/user:	*read character from device*	naming, protection, read,write
operating system:	*character & block API*	hardware specific PIO, interrupt handling, or DMA
hardware:	*keyboard, mouse, etc.*	

Devices

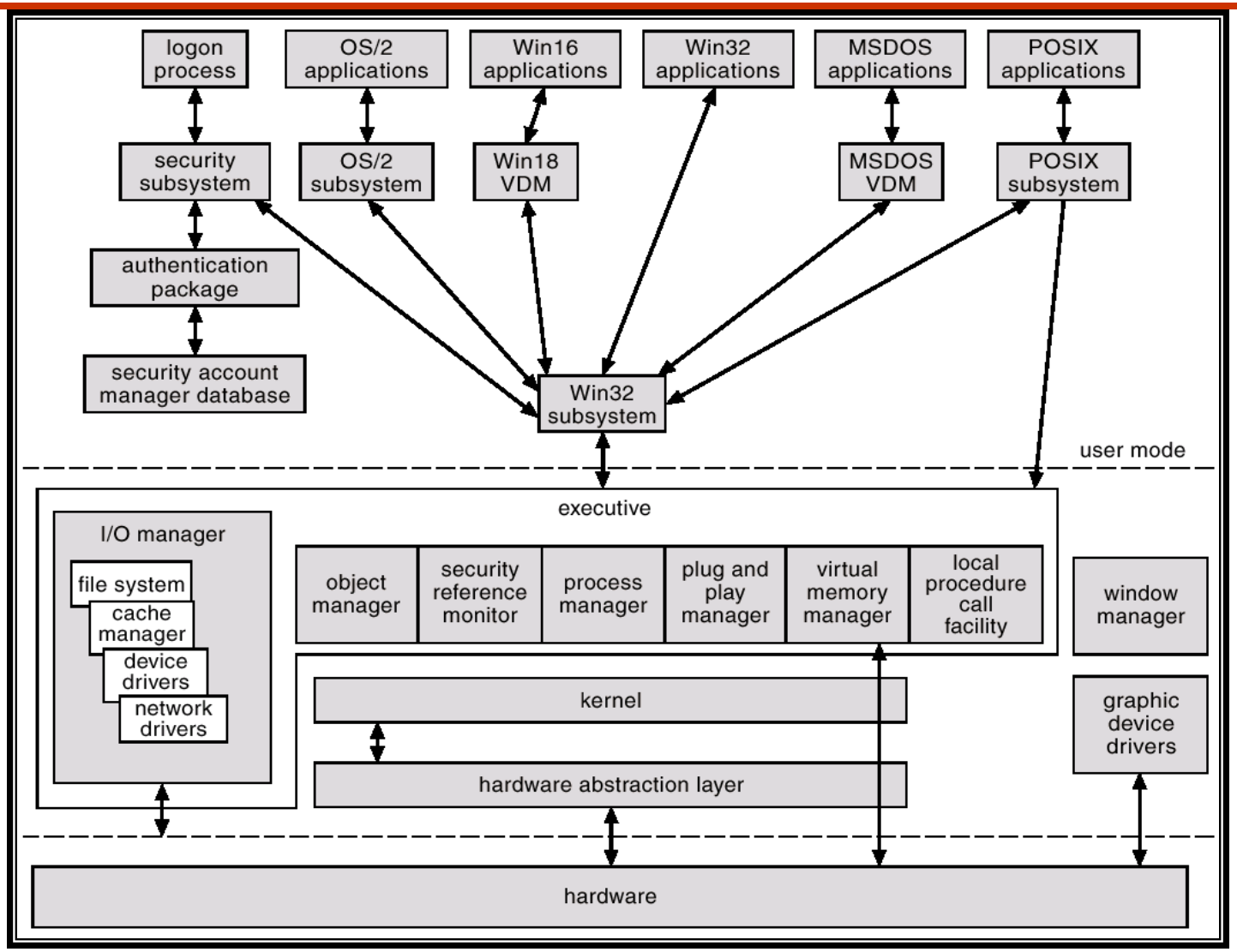
Mechanisms

- Open, close, read, write, ioctl, etc.
- Buffering

Policies

- Protection
- Sharing?
- Scheduling?

Windows 2000 Block Diagram



Major Issues In OS Design

Programming API: what should the VM look like?

Resource management: how should the hardware resources be multiplexed among multiple users?

Sharing: how should resources be shared among multiple users?

Security: how to protect users from each other? How to protect programs from each other? How to project the OS from applications and users?

Communication: how can applications exchange information?

Structure: how to organize the OS?

Concurrency: how do we deal with the concurrency that's inherent in OS'es?

Major Issues In OS Design

Performance: how to make it all run fast?

Reliability: how do we keep the OS from crashing?

Persistence: how can we make data last beyond program execution?

Accounting: how do we keep track of resource usage?

Distribution: how do we make it easier to use multiple computers in conjunction?

Scaling: how do we keep the OS efficient and reliable as the imposed load and so the number of computers grow?

Brief OS History

In the beginning, there really wasn't an OS

- Program binaries were loaded using switches
- Interface included blinking lights (cool!)

Then came batch systems

- OS was implemented to automatically transfer control from one job to the next
- OS was always resident in memory
  - Resident monitor
- Operator provided machine/OS with a stream of programs with delimiters
  - Typically, input was a card reader back then so delimiters were known as control cards

Spooling

CPUs were much faster than card readers and printers

Disks were invented – disks were much faster than card readers and printers

So, what do we do? Indirect and pipeline … what else?

> Read job 1 from cards to disk. Run job 1 while reading job 2 from cards to disk; save output of job 1 to disk. Print output of job 1 while running job 2 while reading job 3 from cards to disk. And so on …
>
> This is known as spooling: Simultaneous Peripheral Operation On-Line
>
> The use of acronyms were alive and well even back then

Can use multiple card readers and printers to keep up with CPU if needed

Improves both system throughput and response time

Multiprogramming

CPUs were still idle whenever executing program needs to interact with peripheral device

- Reading more data from tape

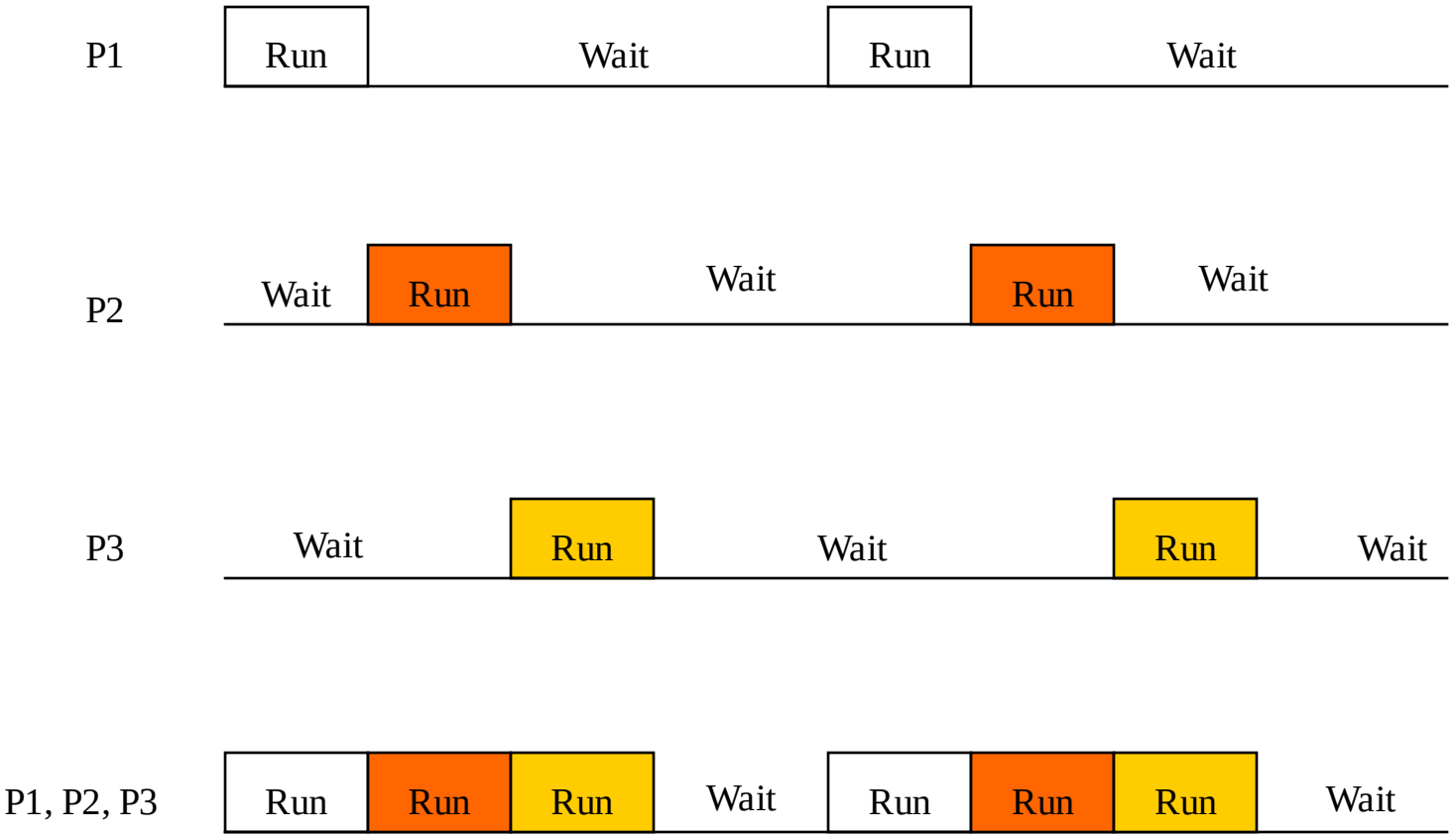
Multiprogrammed batch systems were invented

- Load multiple programs onto disk at the same time (later into memory)
- Switch from one job to another if when the first job performs an I/O operation
  - Peripheral being used had better be slower than disk (or memory)
- Overlap I/O of one job with computation of another job
- Peripherals have to be asynchronous
- Have to know when I/O operation is done: interrupt vs. polling

Increase system throughput, possibly at the expense of response time

- When is this better for response time? When is it worse for response time?

Multiprogramming



Time-Sharing

As you can imagine, batching was a big pain

- You submit a job, you twiddle your thumbs for a while, you get the output, see a bug, try to figure out what went wrong, resubmit the job, etc.
- Even running production programs was difficult in this environment

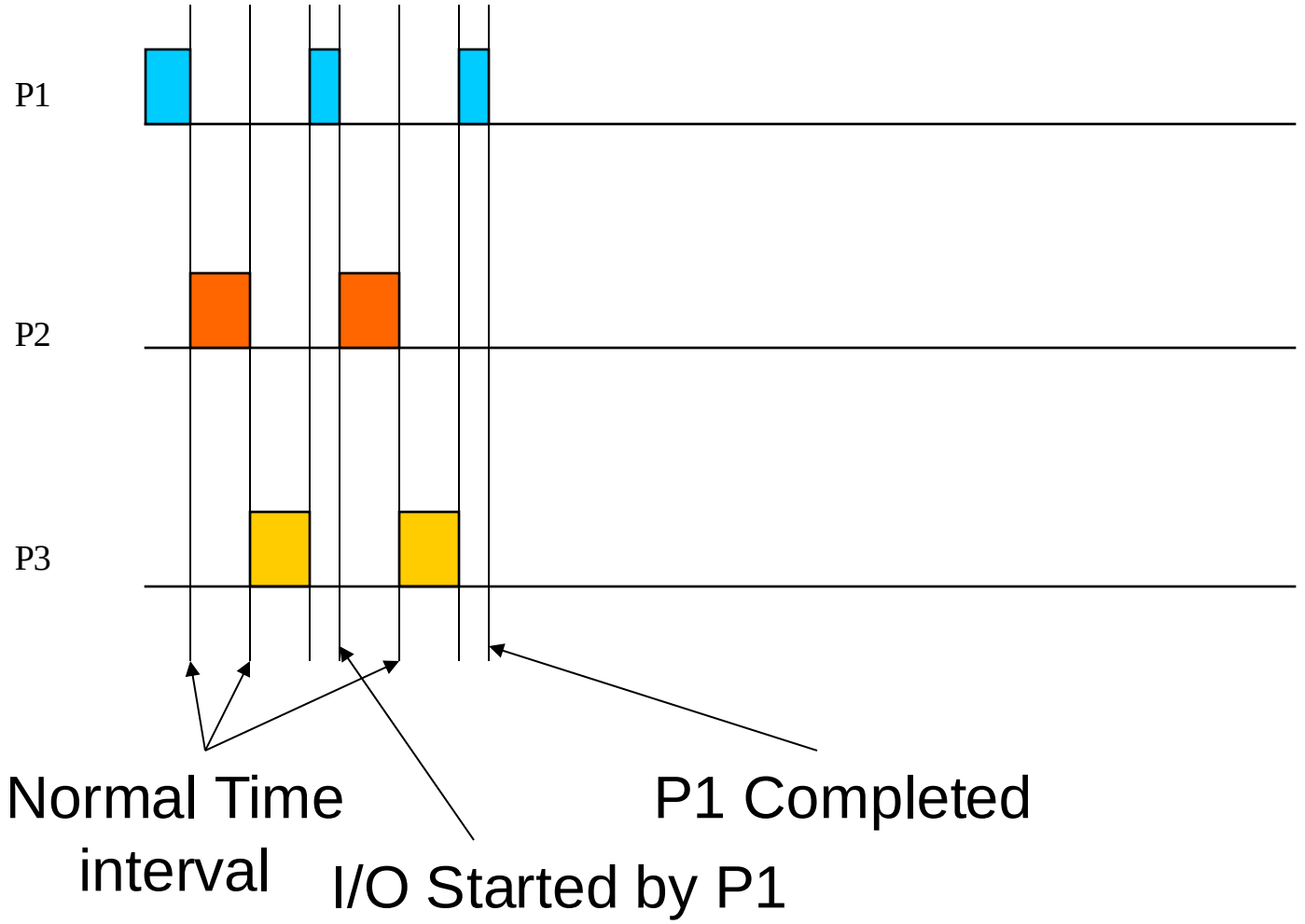
Technology got better: can now have terminals and support interactive interfaces

How to share a machine (remember machines were expensive back then) between multiple people and still maintain interactive user interface?

Time-sharing

- Connect multiple terminals to a single machine
- Multiplex machine between multiple users
- Machine has to be fast enough to give illusion that each user has his/her own machine
- Multics was the first large time-sharing system – mid-1960's

Time-sharing



Parallel OS

Some applications are comprised of tasks that can be executed simultaneously

Weather prediction, scientific simulations, recalculation of a spreadsheet

Can speedup execution by running these tasks in parallel on many processors

Need OS and language support for dividing programs into multiple parallel activities

Need OS support for fast communication and synchronization

Many different parallel architecture

Want performance, performance, and portability

Real-Time OS

Running applications have time deadlines by when they have to complete certain tasks

Hard real-time system

- Medical imaging systems, industrial control systems, etc.
- Catastrophic failure if miss a deadline
  - What happens if collision avoidance software on an oil tanker does not detect another ship before the "turning or breaking" distance of the tanker?
- Challenge lies in how to meet deadlines with minimal resource waste

Soft real-time system

- Multimedia applications
- May be annoying but is not catastrophic if a few deadlines are missed
- Challenge lies in how to meet most deadlines with minimal resource waste
- Challenge also lies in how to load-shed if become overloaded

Distributed OS

Clustering

- Use multiple small machines to handle large service demands
  - Cheaper than using one large machine
  - Better *potential* for reliability, incremental scalability, and absolute scalability

Wide area distributed systems

- Allow use of geographically distributed resources
  - E.g., use of a local PC to access web services
  - Don't have to carry needed information with us

Need OS support for communication and sharing of distributed resources

- E.g., network file systems

Want performance (although speedup is not metric of interest here), high reliability, and use of diverse resources

Embedded OS

Pervasive computing

Right now, cell phones and PDAs

Future, computational elements everywhere

Characteristics

Constrained resources: slow CPU, small memories, no disk, etc.

What's new about this? Isn't this just like the old computers?

Well no, because we want to execute more powerful programs than we use to

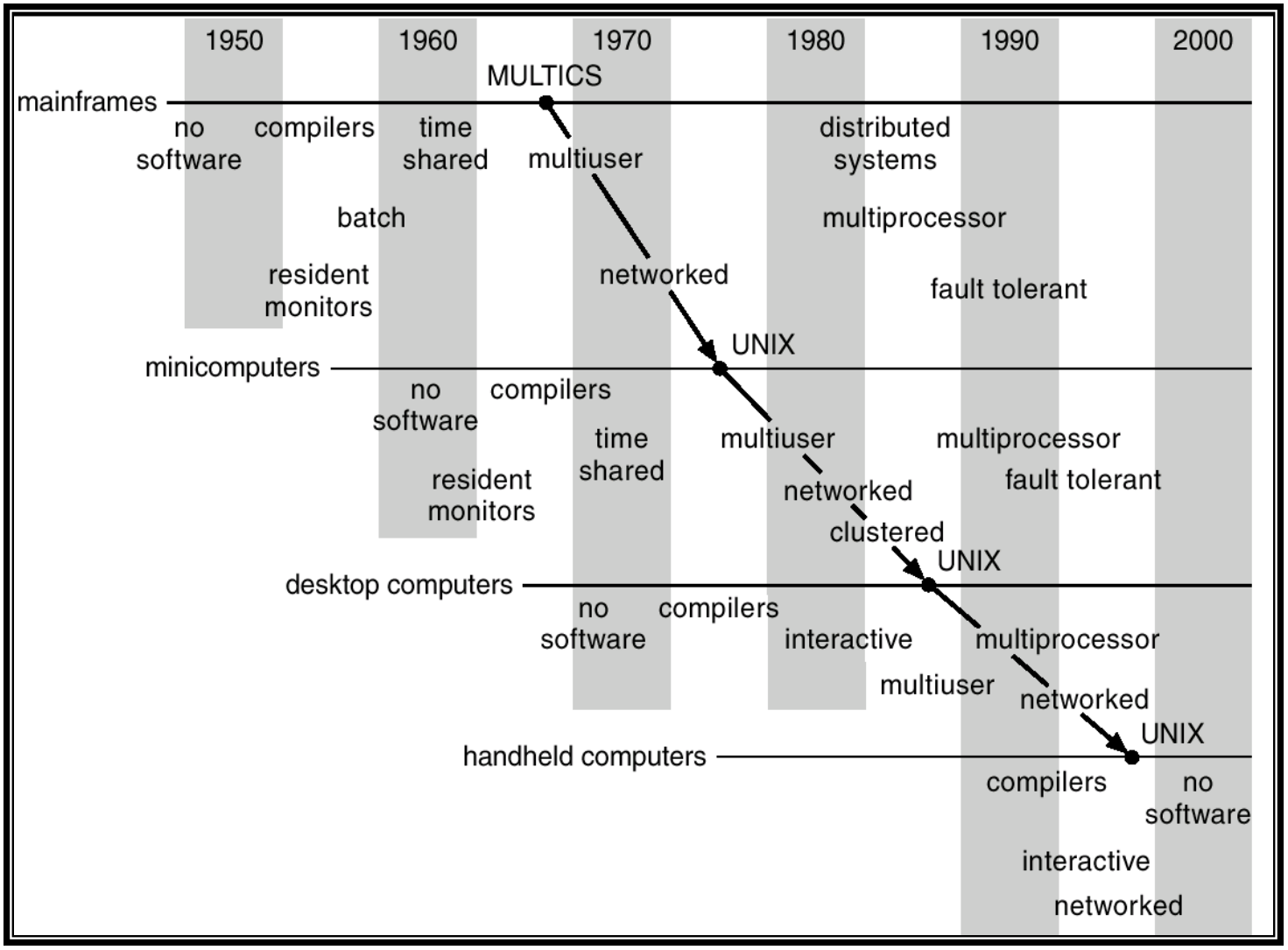
How can we execute more powerful programs if our hardware is similar to old hardware?

Use many, many of them

Augment with services running on powerful machines

OS support for power management, mobility, resource discovery, etc.

Migration of OS Concepts and Features



Next Time

Architectural refresher

Read Silberschatz Chapter 2